

Pràctica 1 - Serial USB

ÍNDEX

L'objectiu de la pràctica és:

Desenvolupar un programa en ESP32 capaç de comunicar-se amb un ordinador a través del port sèrie USB, rebre ordres textuais i controlar un LED, aplicant conceptes bàsics de C++ i eines de gestió de projectes amb GitHub.

0.1 Què és una classe?

Una **classe** és una plantilla o model que defineix les característiques i comportaments que tindran els objectes.

0.2 Què és un objecte?

Un **objecte** és una instància concreta d'una classe. És a dir, és la materialització de la plantilla, un element real amb existència dins del programa.

0.3 Què és un mètode?

Un **mètode** és una funció que pertany a una classe. Defineix el comportament o les accions que pot realitzar un objecte d'aquesta classe.

Relació amb l'objecte **Serial**

En Arduino i ESP32, **Serial** és un objecte que el sistema ja crea automàticament abans que comenci el programa. Aquest objecte forma part de la classe **HardwareSerial**.

Això significa que nosaltres podem utilitzar directament funcions com:

```
Serial.begin(115200);  
Serial.println("Hola");
```

sense necessitat de crear l'objecte manualment.

Comparació amb l'exemple del **Cotxe**

Concepte	Exemple Cotxe	Exemple Serial
Classe	Cotxe (plantilla)	HardwareSerial (plantilla)
Objecte	meuCotxe (instància)	Serial (instància concreta)
Mètodes	encendre(), apagar()	begin(), println(), available(), read()

Diferència principal

Quan treballem amb la classe **Cotxe**, nosaltres hem de crear l'objecte:

```
Cotxe meuCotxe;
```

En canvi, amb **Serial** això no és necessari, perquè Arduino ja proporciona aquest objecte preparat i inicialitzat pel sistema. Per això només hem de començar a utilitzar els seus mètodes.

En aquesta captura es pot observar el codi que hem fet servir.



The image shows a screenshot of an Arduino IDE window. The title bar indicates the file is named 'sketch_may15a'. The code editor contains the following C++ code:

```
// Definir el pin LED
const int LED_PIN = 17;

void setup() {
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW); // LED apagado inicialmente

  Serial.begin(115200);

  delay(1000);
  Serial.println("Sistema listo. Escribe ON u OFF para controlar el LED");
}

void loop() {
  if (Serial.available()) {
    String linea = Serial.readStringUntil('\n');
    linea.trim();

    Serial.print("Recibido: ");
    Serial.println(linea);

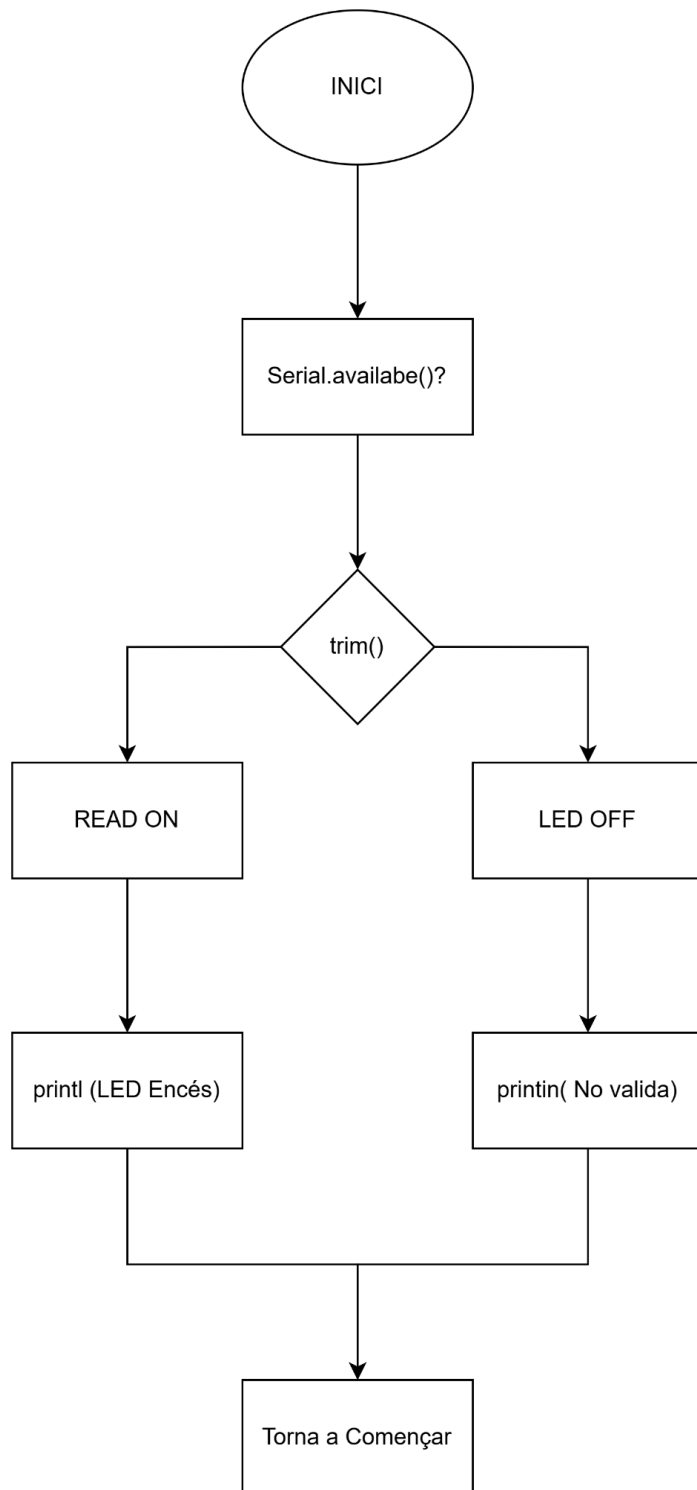
    if (linea.equalsIgnoreCase("ON")) {
      digitalWrite(LED_PIN, HIGH);
      Serial.println("LED ENCENDIDO");
    }
    else if (linea.equalsIgnoreCase("OFF")) {
      digitalWrite(LED_PIN, LOW);
      Serial.println("LED APAGADO");
    }
    else if (linea.length() > 0) {
      Serial.println("Comando desconocido");
    }
  }
}
```

To the right of the code editor is the serial monitor window. It shows the output: 'Sistema listo. Escribe ON u OFF para controlar el LED'. At the bottom of the serial monitor, there are two checkboxes: 'Autoscroll' (checked) and 'Mostrar marca temporal' (unchecked).

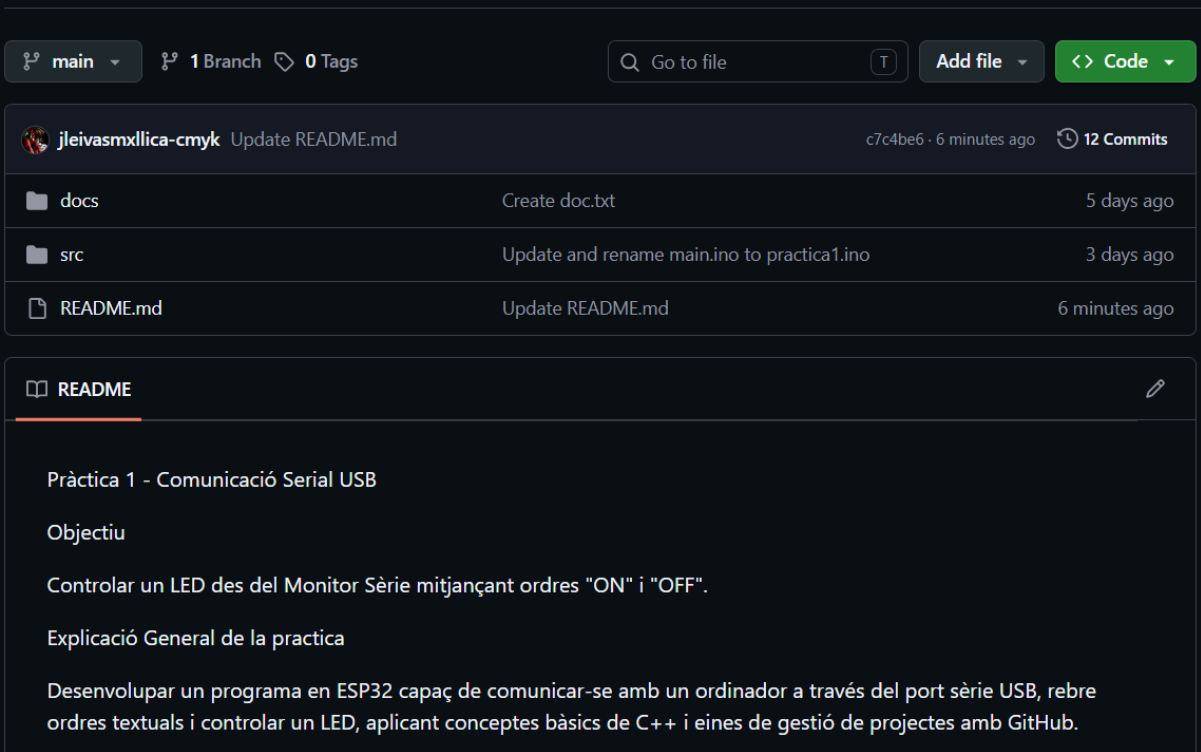
Video:

VID_20260515_141208.mp4

Diagrama de Flux:



Seguiment del projecte dins de Github. Amb imatges reals del vostre projecte a GitHub cal explicar tot el procés (repositori, taulell Kanban, etc...)



The screenshot shows a GitHub repository interface. At the top, there's a navigation bar with 'main' branch selected, '1 Branch', and '0 Tags'. A search bar 'Go to file' and buttons 'Add file' and 'Code' are also visible. Below the navigation bar, the repository name 'jleivasmxllica-cmyk' is shown, followed by the commit message 'Update README.md' and the commit hash 'c7c4be6' with the time '6 minutes ago' and '12 Commits'. A list of files is displayed: 'docs' (Create doc.txt, 5 days ago), 'src' (Update and rename main.ino to practica1.ino, 3 days ago), and 'README.md' (Update README.md, 6 minutes ago). Below the file list, the 'README' file is selected and its content is shown. The README content includes the title 'Pràctica 1 - Comunicació Serial USB', the objective 'Objectiu', the task description 'Controlar un LED des del Monitor Sèrie mitjançant ordres "ON" i "OFF".', and the general explanation 'Explicació General de la practica'. The final paragraph describes the project goal: 'Desenvolupar un programa en ESP32 capaç de comunicar-se amb un ordinador a través del port sèrie USB, rebre ordres textuais i controlar un LED, aplicant conceptes bàsics de C++ i eines de gestió de projectes amb GitHub.'

main 1 Branch 0 Tags

Go to file Add file Code

jleivasmxllica-cmyk Update README.md c7c4be6 · 6 minutes ago 12 Commits

docs	Create doc.txt	5 days ago
src	Update and rename main.ino to practica1.ino	3 days ago
README.md	Update README.md	6 minutes ago

README

Pràctica 1 - Comunicació Serial USB

Objectiu

Controlar un LED des del Monitor Sèrie mitjançant ordres "ON" i "OFF".

Explicació General de la practica

Desenvolupar un programa en ESP32 capaç de comunicar-se amb un ordinador a través del port sèrie USB, rebre ordres textuais i controlar un LED, aplicant conceptes bàsics de C++ i eines de gestió de projectes amb GitHub.

 main practica1 / src / practica1.ino 

 **jleivasmxllca-cmyk** Update and rename main.ino to practica1.ino

Code **Blame**

34 lines (28 loc) · 773 Bytes 

```
1 // Definir el pin LED
2 const int LED_PIN = 17;
3
4 void setup() {
5     pinMode(LED_PIN, OUTPUT);
6     digitalWrite(LED_PIN, LOW); // LED apagado inicialmente
7
8     Serial.begin(115200);
9
10    delay(1000);
11    Serial.println("Sistema listo. Escribe ON u OFF para controlar el LED");
12 }
13
14 void loop() {
15     if (Serial.available()) {
16         String linea = Serial.readStringUntil('\n');
17         linea.trim();
18
19         Serial.print("Recibido: ");
20         Serial.println(linea);
21
22         if (linea.equalsIgnoreCase("ON")) {
23             digitalWrite(LED_PIN, HIGH);
24             Serial.println("LED ENCENDIDO");
25         }
26         else if (linea.equalsIgnoreCase("OFF")) {
27             digitalWrite(LED_PIN, LOW);
28             Serial.println("LED APAGADO");
29         }
```